RDFQuotient documentation

June 18, 2021

1 RDFQuotient

The RDFQuotient tool, hosted at Inria GitLab repository[1], is an implementation of the framework for computing quotient summaries of RDF graphs[2]. The current version of the software is 1.9. A link to download a stand-alone jar file and building-from-sources instructions can be found in README file[3]. RDFQuotient uses PostgreSQL DBMS as a back-end and it is a requirement for the setup. PostgreSQL server must be running with the specified configuration. For docker instructions, please refer to the README file.

2 Command-line, stand-alone application interface

The command-line interface consists of three possible operations: `load`, `summarize`, and `read`. Even though `read` operation is present in the command-line interface, it doesn't have any effect and was added for completeness. It is meant to be used programmatically. The operations follow a specific syntax that can be seen in the RDFQuotient manual in Figure 1.

Specifying `dataset.filename` argument for `--load` operation is the only way to change the input graph filename. The `--summarize` operation has two possible settings: `dataset.filename` and `database.name`. Specifying one of them is mandatory. However, if both of them are present `database.name` takes precedence. The reason for keeping the two options is that sometimes it is more convenient to use the same name for the argument, especially when we rely on the default naming of the databases created for summaries.

Example command-line execution for loading:

```
java -jar target/RDFQuotient-1.8-with-dependencies.jar --load
"dataset.filename=/data/datasets/enelshops/enelshops.nt"
```

Example command-line execution for summarization in a dry-run mode:

```
java -jar target/RDFQuotient-1.8-with-dependencies.jar --summarize
"dataset.filename=/data/datasets/enelshops/enelshops.nt" --dry-run
```

Example command-line execution for summarization:

```
java -jar target/RDFQuotient-1.8-with-dependencies.jar --summarize
"dataset.filename=/data/datasets/enelshops/enelshops.nt"
```

2.1 Configuration

The tool encapsulates hard-coded values for configuration that model all the available functionalities and scenarios. They can be overwritten by command-line arguments or by two configuration files: `loading.properties` and `summarization.properties`. The command-line arguments and configuration files are optional. The configuration files serve the purpose of templates, they are filled in with the default, hard-coded values.

[1] `https://gitlab.inria.fr/cedar/RDFQuotient`
[2] `https://project.inria.fr/rdfquotient/`
[3] `https://gitlab.inria.fr/cedar/RDFQuotient/blob/master/README.md`

```
$ ./RDFQuotient.sh --help
usage: RDFQuotient 1.9

This framework is designed to work with one graph at a time.
Before using RDFQuotient make sure that the Postgres server is running.
Input RDF dataset file format is N-Triples and the file is assumed not to
contain any duplicated triples.

rdfquotient [-d] [-h | -l <[ARGS]> | -r <[ARGS]> | -s <[ARGS]> | -v]  [-lp
       <filename> | -sp <filename>]
-d,--dry-run                                        - print the configuration used
                                                    for a run
-h,--help                                           - print help message
-l,--load <[ARGS]>                                  - load the dataset into database
                                                    [ARGS] must specify a value for
                                                    dataset.filename
                                                    CAUTION: database is dropped by
                                                    default
-lp,--use-loading-properties <filename>             - set loading properties
                                                    filename
-r,--read <[ARGS]>                                  - read summary from database
                                                    [ARGS] must specify a value for
                                                    database.name
-s,--summarize <[ARGS]>                             - summarize the dataset
                                                    [ARGS] must specify a value for
                                                    dataset.filename or
                                                    database.name
-sp,--use-summarization-properties <filename>       - set summarization properties
                                                    filename
-v,--version                                        - print the version of
                                                    RDFQuotient

[ARGS] is a comma-separated list of assignments of form key=value, where key is a
configuration property from the list of loading or summarization configuration
properties.
```

Figure 1: Command-line interface of RDFQuotient

To determine the configuration for a run of the tool for either loading or summarization properties files, Algorithm 1 applies. Hard-coded values have the lowest priority. Values present in the configuration files have medium priority. Values passed in the arguments have the highest priority.

Algorithm 1 Setting up of the configuration

1: **procedure** SETUPCONFIGURATION
2: **if** `args["properties_filename"]` $\neq$ `null` **then**
3: `properties_filename` $\leftarrow$ `args["properties_filename"]`
4: **if** `properties_filename` file exists **then**
5: overwrite the default, hard-coded values for `properties` with the values in `properties_filename` file
6: **for each** `setting` $\in$ `args.values` **do**
7: overwrite the value of `setting` in `properties` with the value specified by `args[setting]`

2.2 Configuration files

The following two lists contain all the functionalities and scenarios in terms of the configuration parameters of the execution.

2.2.1 Loading properties

The list of loading configuration properties with default values assigned:

1. `dataset.filename = test.nt`
   Caution: RDFQuotient tool assumes the N-Triples format for the dataset file and no duplicates present.
2. `database.host = localhost`
3. `database.port = 5432`
4. `database.user = postgres`
5. `database.password = postgres`
6. `database.name =`
   Blank by default.
   If left blank, the database name will be derived from the dataset filename. Otherwise, the specified name will be used.
7. `database.drop_existing_db = true`
   Valid options: `true`, `false`.
   Caution: if the `database.name` exists, it is dropped by default before loading.
8. `database.storage_layout = TRIPLES_TABLE`
   Valid options: `TRIPLES_TABLE`, `TABLE_PER_ROLE_AND_CONCEPT`.
9. `database.triples_table_name = triples`
10. `database.encoded_triples_table_name = encoded_triples`
11. `database.encoded_saturated_triples_table_name = encoded_saturated_triples`
12. `database.dictionary_table_name = dictionary`
13. `dictionary.fetch_size = 1000`
14. `saturation.type`
    Which saturation algorithm to use while loading the graph. Valid options: `NONE`, `CONSTRAINT_SAT`, `ASSERTION_SAT`, `RDFS_SAT`.
15. `saturation.enable = false`
    Valid options: `true`, `false`. Warning: this is an old option. The `true` value corresponds to the `ASSERTION_SAT` algorithm. The `saturation.type` option has higher priority.
16. `database.encoded_saturated_triples_table_name = encoded_saturated_triples`
17. `saturation.batch_size = 1000`
18. `database.cluster_indexes=true`
    Whether to add clustered indexes to the tables created by OntoSQL (enabling this is recommended for better performance).
19. `database.deterministic_ordering = false`
    Whether to sort dictionary entries before assigning encodings and also`encode_triples` and `encoded_saturated_triples` tables. Used for tests, should not be used for big graphs.

20. `statistics.export_to_csv_file = true`
    Valid options: `true`, `false`.

21. `configuration.export_to_disk = false`
    Whether to export `loading.properties` used in a run after it's finished.
    Valid options: `true`, `false`.

2.2.2 Summarization properties

The list of summarization configuration properties with default values assigned:

1. `dataset.filename = test.nt`

2. `database.host = localhost`

3. `database.port = 5432`

4. `database.user = postgres`

5. `database.password = postgres`

6. `database.name =`
    Blank by default.
    If left blank, the database name will be derived from the dataset filename. Otherwise, the specified name will be used.

7. `database.triples_table_name = triples`

8. `database.encoded_triples_table_name = encoded_triples`

9. `database.dictionary_table_name = dictionary`

10. `database.encoded_saturated_triples_table_name = encoded_saturated_triples`

11. `database.deterministic_ordering = false`
    Whether to sort the `encoded_triples` and `encoded_saturated_triples` tables. Used for tests, should not be used for big graphs.

12. `summary.summarize_saturated_graph = false`
    Raises an error in case of absence of saturated triples table.
    Valid options: `true`, `false`.

13. `summary.type = typedweak`
    Valid options: `weak`, `strong`, `typedweak`, `typedstrong`, `2pweak`, `2pweakunionfind`, `2pstrong`, `2ptypedweak`, `2ptypedstrong`, `onefb`, `onefw`.

14. `summary.omit_generic_properties_from_cliques = true`
    Valid options: `true`, `false`.

15. `summary.generic_properties = <http://www.w3.org/2000/01/rdf-schema#label>,`
    `<http://www.w3.org/2000/01/rdf-schema#comment>,`
    `<http://www.w3.org/1999/02/22-rdf-syntax-ns#value>,`
    `<http://www.w3.org/2002/07/owl#sameAs>`
    List of comma-separated values of URI within angle brackets.

16. `summary.replace_type_with_most_general_type = true`
    Raises an error in the case of the `W` and `S` summaries.
    Valid options: `true`, `false`.

17. `summary.consistency_checks = false`
    This may make summarization significantly slower; set it to true only for debugging.
    Valid options: `true`, `false`.

18. `summary.export_to_nt_file = true`
    Valid options: `true`, `false`.

19. `summary.nt_file_prefix = summariesNT/`
    Prefix added to summary NT file, usually to dispatch it into a separate directory.

20. `summary.export_to_database = true`
    Valid options: `true`, `false`.

21. `summary.save_representation_function_and_node_statistics = true`
    Valid options: `true`, `false`.

22. `summary.gather_representation_counts = true`
    Valid options: `true`, `false`.

23. `drawing.draw_input_graph = false`
    Whether to draw input RDF graph.
    Valid options: `true`, `false`.

24. `summary.step_by_step = false`
    Whether to execute step-by-step summarization. If true, then it is recommended to set `drawing.step_by_step = always`.

25. `statistics.export_to_csv_file = true`
    Valid options: `true`, `false`.

26. `configuration.export_to_disk = false`
    Valid options: `true`, `false`.
    Whether to export `summarization.properties` used in a run after it's finished.

27. `drawing.dot_installation = /usr/local/bin/dot`
    Path to dot executable. If this is not found, it is not an error, just drawing won't work.

28. `drawing.remove_dot_file = false`
    Whether to remove DOT file in case of successful execution of dot.

29. `drawing.dot_file_prefix = summariesDOT/`
    Prefix added to the summary DOT file, usually to dispatch it into a separate directory.

30. `drawing.png_file_prefix = summariesPNG/`
    Prefix added to the summary PNG file, usually to dispatch it into a separate directory.

31. `drawing.step_by_step = false`
    Valid options: `false`, `always`, `when_changes`.
    Whether to draw step-by-step drawings.

32. `drawing.style = split_and_fold_leaves`
    Valid options: `plain`, `split_leaves`, `split_and_fold_leaves`.
    If an invalid option is specified, the drawing will be suppressed.

33. `drawing.color_scheme = diverse`
    Valid options: `diverse`, `bw`, `ivory`, `light`.

34. `drawing.summary_node_URI_prefix = http://rq.org/`

35. `drawing.title = true`
    Valid options: `true`, `false`.

36. `drawing.max_node_label_length = 20`

37. `drawing.max_types_drawn_per_namespace = 5`

38. `drawing.summary_node_support_URI_prefix = http://rq.org/nodeSupport`
39. `drawing.summary_edge_support_URI_prefix = http://rq.org/edgeSupport`
40. `drawing.reified_summary_edge_URI_prefix = http://rq.org/edge`
41. `drawing.reified_edge_subject_URI_prefix = http://rq.org/reifiedEdgeSubject`
42. `drawing.reified_edge_property_URI_prefix = http://rq.org/reifiedEdgeProperty`
43. `drawing.reified_edge_object_URI_prefix = http://rq.org/reifiedEdgeObject`

3 Programmatic Java interface

The same set of options, as for stand-alone interface, is also available for programmatic Java interface: `fr.inria.cedar.RDFQuotient.controller.Interface` class.

Here, instead of passing command-line arguments to the program, we pass a `Properties` Java object to one of the dedicated methods. We can explicitly set values of some configuration properties using other dedicated methods. We can also access the `Connection` Java object and not close it if we don't want to. It makes this implementation stateful as it can potentially use a connection opened outside of this class and can return it to be used later. The command-line implementation is stateless.

See detailed explanation of methods in Interface class below:

1. `public static Connection getOrEstablishNewDatabaseConnection(Properties properties)`
   If the database connection is established it returns it, otherwise establishes a new connection using the configuration specified in `properties`.
2. `public static Connection getDatabaseConnection()`
   Returns database connection, may return `null` if database connection was not established or closed.
3. `public static void closeDatabaseConnection()`
   Closes database connection.
4. `public static void forceCloseDatabaseConnection(Properties properties)`
   This is an emergency method if some library misbehaves and doesn't close the connection, should not be used otherwise.
5. `public static String load(String configurationFilename, Properties properties, boolean closeConnection)`
   If both configurationFilename and properties are null or point to a file/object that does not contain valid configuration properties, default values will be used.
   Otherwise, the following procedure applies:
   (a) Take default values
   (b) Overwrite them with the values from the `configurationFilename` file
   (c) Overwrite them with the values from the object properties

   Parameter closeConnection controls whether to close connection to the database after the call to this method.
   Examples:
   (a)
   ```
   Properties myProperties = new Properties;
   myProperties.put("dataset.filename", "datasets/my_dataset.nt");
   Interface.load(LoadingProperties.getDefaultLoadingPropertiesFilename(),
   myProperties, true);
   ```
   (b)
   ```
   Properties myProperties = LoadingProperties.getDefaultProperties();
   myProperties.put("dataset.filename", "datasets/my_dataset.nt");
   Interface.load(null, myProperties, true);
   ```

(c)
```
Properties myProperties = LoadingProperties.getDefaultProperties();
myProperties.put("dataset.filename", "datasets/my_dataset.nt");
Interface.load("conf/my_config_file.properties", myProperties, true);
```

Returns a name of the database.

6. `public static HashMap<String, String> summarize(String configurationFilename, Properties properties, boolean closeConnection)`
   See load method comment: in examples use SummarizationProperties class instead of LoadingProperties.
   Returns a map with keys: `"databaseName"`, `"NTFilename"` and `"DOTFilename"`. If the corresponding name is not present, the value is set to null.

7. `public static Summary read(String configurationFilename, Properties properties, boolean closeConnection)`
   See load method comment.
   Returns Summary object.

8. `public static HashMap<String, String> loadAndSummarize(Properties loadingProperties, Properties summarizationProperties)`
   For convenience, this method combines load and summarize functionalities in one. By default, the database connection is closed.

9. `public static HashMap<String, String> loadAndSummarizeThroughShortcut(Properties loadingProperties, Properties summarizationProperties)`
   For convenience, this method combines load and summarize through shortcut functionalities in one. By default, the database connection is closed.

4 Tests

Tests are stored in the `src/test` directory. Resources are stored in the `src/test/resources` directory that contains all the input test files for our automatic correctness tests (not to confuse with `consistencyChecks` which are tests that can be performed during the execution of the program depending on the configuration). They are checked once the `mvn install` command is run. They can be skipped by adding a flag `-DskipTests`, e.g., running the `mvn clean install -DskipTests` command.

5 Scripts

Project tree contains `scripts` directory with useful bash scripts for computing the summaries with different configurations specified. These scripts account for memory settings of the Java Virtual Machine, as well as they can realize shortcut computations using a stand-alone interface by composing a pipeline of commands.